CS527 – Computer Systems
Lab 9: OpenMP

Task 1: Learn recursive task parallelism using `#pragma omp task` and `#pragma omp taskwait`. Sort an array of integers in ascending order using merge sort. Parallelize the recursive sorting of the left and right halves.

Requirements:

1. Implement a serial merge sort.
2. Implement a parallel merge sort using OpenMP tasks.
3. Use a cutoff threshold so that small subarrays are sorted serially rather than creating too many tasks.
4. Verify that the parallel output is sorted and matches the serial output.
5. Test array sizes of $10^5, 10^6, 10^7$ using 1, 2, 4, and 8 threads.
6. Report execution time, speedup, and parallel efficiency.

Sample code for the recursive parallel portion:

```c
#include <omp.h>

void merge_sort_parallel(int *a, int *tmp, int left, int right) {
      if (left >= right)
      return;

      if (right - left < 2048) {
      merge_sort_serial(a, tmp, left, right);
      return;
      }

      int mid = left + (right - left) / 2;

      #pragma omp task shared(a, tmp) firstprivate(left, mid)
      merge_sort_parallel(a, tmp, left, mid);

      #pragma omp task shared(a, tmp) firstprivate(mid, right)
      merge_sort_parallel(a, tmp, mid + 1, right);

      #pragma omp taskwait
      merge(a, tmp, left, mid, right);
}
```

Call the recursive function inside a parallel region:

```c
#pragma omp parallel
{
      #pragma omp single
      merge_sort_parallel(a, tmp, 0, n - 1);
```

```
}
```

Here, merge_sort_serial() and merge() are helper functions that you must implement. The temporary array must be allocated before sorting and shared safely across tasks. Each recursive call must operate on a disjoint subarray, and the parent must wait for both child tasks before merging.

Questions to address in your report:

- What is the difference between OpenMP tasks and parallel for?
- Why is taskwait required before merging the two halves?
- How does the cutoff threshold affect performance?
- Why can parallel merge sort become slower than serial merge sort for small arrays?
- How does task creation overhead affect scalability?

Task 2: Understand how `static, dynamic,` and `guided` scheduling affect load balancing and performance.

Given an array of N elements, perform a computationally expensive operation on each element. Make the workload uneven so that some iterations take significantly longer than others.

Requirements:

1. Implement a serial version as the baseline.
2. Implement parallel versions using:
    - `schedule(static)`
    - `schedule(dynamic, chunk_size)`
    - `schedule(guided)`
3. Test with $N=10^5,10^6,10^7$
4. Measure execution time using 1, 2, 4, and 8 threads.
5. Compare speedup and parallel efficiency for each scheduling strategy.

Sample code:

c

```c
#include <stdio.h>
#include <omp.h>

#define N 1000000

int main(void) {
      static double a[N];

      for (int i = 0; i < N; i++)
      a[i] = i * 0.001;

      double start = omp_get_wtime();

      #pragma omp parallel for schedule(dynamic, 100)
      for (int i = 0; i < N; i++) {
      double x = a[i];

      // Simulate an uneven computational workload.
      int work = (i % 1000) * 100;
      for (int j = 0; j < work; j++)
            x = x * 1.0000001 + 0.0000001;
```

```
        a[i] = x;
        }

        double end = omp_get_wtime();
        printf("Execution time: %f seconds\n", end - start);

        return 0;
}
```

Questions to address in your report:

- What is the difference between static, dynamic, and guided scheduling?
- Which scheduling strategy performs best for uniform workloads? What about uneven workloads?
- Why can dynamic scheduling introduce additional overhead?
- How does the chunk size affect performance?

Task 3: Practice multiple reductions in a single parallel loop.
Given an integer array a[N], count the number of even and odd elements.

Requirements:

1. Implement a serial solution.
2. Implement a parallel solution using `reduction(+:even_count, odd_count)`.
3. Verify that the parallel counts match the serial counts.
4. Test with $N=10^5,10^6,10^7,10^8$
5. Measure execution time using 1, 2, 4, and 8 threads.
6. Compare the reduction-based solution with a version that uses `#pragma omp atomic` to update the counters.

Sample code:

```
c
void count_even_odd(int *a, long n,
                    long *even_count, long *odd_count) {
        long even = 0, odd = 0;

        #pragma omp parallel for reduction(+:even, odd)
        for (long i = 0; i < n; i++) {
        if (a[i] % 2 == 0)
                even++;
        else
                odd++;
        }

        *even_count = even;
        *odd_count = odd;
}
```

Questions to address in your report:

- How can multiple variables be included in a reduction clause?
- Why is the reduction approach generally faster than using atomic updates?
- Why must the sum of the even and odd counts equal N?
- Does increasing the number of threads always improve performance?